

# **Academia de Studii Economice din București**

**Facultatea de Cibernetică, Statistică și Informatică Economică**

## **PROIECT: REȚELE DE CALCULATOARE**

**Sistem de Licitatii în Timp Real bazat pe TCP**

**Covrig Eduard-Gabriel**

**Constantin Arthur-Stefan**

**Ciobanu Cătălin-Marian**

# 1. Introducere

Proiectul constă în dezvoltarea unei aplicații distribuite de tip client-server, care simulează un mediu de licitații în timp real. Sistemul permite utilizatorilor să se conecteze concurrent la un server central, să publice obiecte pentru licitație și să plaseze oferte pentru obiectele publicate de alți utilizatori.

Aplicația este construită folosind limbajul Python, comunicarea realizându-se prin intermediul socket-urilor de rețea pe protocolul TCP (Transmission Control Protocol). Pentru a asigura portabilitatea și o instalare facilă, componenta de server a fost containerizată folosind tehnologia Docker.

Scopul principal al proiectului este de a demonstra conceptele fundamentale de rețelistică: comunicarea prin socket-uri, gestionarea conexiunilor concurente (multi-threading), serializarea datelor (JSON) și sincronizarea accesului la resurse partajate în medii distribuite.

## 2. Arhitectura Aplicației

Aplicația respectă arhitectura clasică **Client-Server**:

- **Serverul:** Acționează ca un nod central. El menține starea globală a sistemului (lista de clienți conectați și lista de produse active), validează ofertele și trimite notificări (broadcast) către toți utilizatorii atunci când apar evenimente noi (publicarea unui produs, plasarea unei oferte mai mari, expirarea unei licitații).
- **Clientul:** Reprezintă interfața utilizatorului. Preia comenzile de la tastatură, le formatează sub formă de mesaje JSON și le trimite serverului. Totodată, ascultă permanent mesajele venite de la server pentru a le afișa în consolă.

### 2.1. Tehnologii și Protocoale Utilizate

- **Protocolul de transport (TCP):** A fost ales TCP deoarece într-un sistem de licitații este critic ca mesajele (ofertele de preț) să ajungă la destinație în mod fiabil, fără pierderi de pachete și în ordinea în care au fost trimise.

- **Formatul datelor (JSON):** Schimbul de mesaje structurate între client și server (de exemplu, comenzile de `publish` sau `bid`) se face formatat în JSON. Acest lucru asigură o parsare ușoară și extensibilitate.
- **Multi-threading:** Folosit atât pe server (pentru a deservi mai mulți clienți simultan) cât și pe client (pentru a putea citi de la tastatură și a asculta mesajele de la server în același timp).
- **Docker:** Folosit pentru a izola mediul de execuție al serverului, expunând doar portul necesar comunicării (8719).

## 3. Detalii de Implementare

### 3.1. Componenta Server (`server.py`)

Serverul este inițializat să asculte pe interfața `0.0.0.0` (pentru a permite conexiuni din afara containerului Docker) și pe portul `8719`.

- **Gestionarea Concurenței:** Pentru fiecare client nou conectat, serverul creează un fir de execuție separat (`threading.Thread`).
- **Sincronizarea (Lock-uri):** Deoarece mai mulți clienți pot licita exact în același timp pentru același produs, se folosește un mecanism de tip `threading.RLock()`.
- **Timere pentru expirare:** Când un produs este publicat, se declanșează un `threading.Timer` setat la 30 de secunde (`AUCTION_TIME`). Când acest timer expiră, funcția `expire_product` marchează produsul ca fiind inactiv și anunță toți clienții cine este câștigătorul.

### 3.2. Componenta Client (`client.py`)

Clientul este conceput să fie asincron din perspectiva utilizatorului.

- **Thread-ul principal:** Preia comenzile utilizatorului (`publish` sau `bid`), le împachetează într-un dicționar, le serializează folosind `json.dumps()` și le trimite serverului prin socket.

- **Thread-ul secundar (asculta\_server):** Rulează în fundal (daemon) într-o buclă infinită, așteptând mesaje de tip broadcast de la server pentru a le printa instantaneu în terminal, fără a bloca promptul utilizatorului.

## 4. Utilizarea Aplicatiei

### Pornirea serverului:

docker build -t server-licitatie .

```
[gr3_19@sys:~/proiect$ docker build -t server-licitatie .  
STEP 1/4: FROM python:3.9-slim  
STEP 2/4: WORKDIR /app  
--> Using cache b29bb0624708ebcf7709d85758efd4ce5617cfc3d107328446ddc7be74fba9bd  
--> b29bb0624708  
STEP 3/4: COPY server.py .  
--> a49767749f94  
STEP 4/4: CMD ["python", "server.py"]  
COMMIT server-licitatie  
--> 59ade1ca9c6c  
Successfully tagged localhost/server-licitatie:latest  
59ade1ca9c6cc5736b4ada192b616b89518665269c489f80b983214bca4794ce
```

docker run -p port:port server-licitatie

```
[gr3_19@sys:~/proiect$ docker run -p 8719:8719 server-licitatie  
]
```

### Conectarea clientilor:

python3 client.py

```
[gr3_19@sys:~/proiect$ python3 client.py  
Introdu numele tau: Covrig Eduard  
Comenzi disponibile:  
  publish <produs> <pret>  
  bid <produs> <pret>  
>  
[SERVER] Te-ai conectat cu succes!  
> ]
```

### Comenzi disponibile pentru utilizatori:

**publish <nume> <pret>** (Produsul devine activ pentru 30 de secunde, iar toți ceilalți clienți sunt notificați)

```
[> publish laptop 100  
>  
[SERVER] NOU! Covrig Eduard vinde 'laptop' cu 100.0 lei.  
> █
```

```
>  
[SERVER] Licitatia pt 'laptop' a expirat! Castigator: Nimeni (Pret: 100.0)  
> █
```

**bid <nume> <pret>** (Serverul verifică dacă oferta este strict mai mare decât prețul curent. În caz afirmativ, prețul este actualizat, iar sistemul trimite un mesaj global despre noua ofertă)

```
[> bid ceas 250  
>  
[SERVER] OFERTA! Covrig Eduard da 250.0 lei pt 'ceas'.  
> █
```

Când expiră cele 30 de secunde, sistemul blochează automat primirea altor oferte pentru produsul respectiv și anunță numele participantului care a oferit cel mai mare preț.

```
>  
[SERVER] Licitatia pt 'ceas' a expirat! Castigator: Covrig Eduard (Pret: 250.0)  
> █
```